

Getting Started

Prerequisites

- Node.js ≥ 18
- PostgreSQL 15 or a free [Supabase](#) project

1. Clone & install

```
git clone https://github.com/Shubham11440/EN2H_Booking_Platform.git
cd EN2H_Booking_Platform
npm install
```

2. Configure environment

```
cp .env.example .env
```

Open `.env` and fill in your values (see [Environment Variables](#) below).

3. Run the database migration

```
npx prisma migrate dev --name init
```

4. Start the dev server

```
npm run start:dev
```

The API is live at **`http://localhost:3000`** Swagger UI is at **`http://localhost:3000/api/docs`**

Running with Docker (fully local)

```
# Start PostgreSQL + app together
docker-compose up --build
```

```
# In another terminal – run migrations against the container
docker-compose exec app npx prisma migrate deploy
```

If you are using Supabase, only run the `app` service and point `DATABASE_URL` in `.env` to your Supabase connection string.

Environment Variables

Variable	Required	Description
PORT	No	HTTP port (default: 3000)
DATABASE_URL		Prisma connection URL (pooler for runtime)
DIRECT_URL	Supabase only	Direct connection URL used by

Variable	Required	Description
		<code>prisma migrate</code>
JWT_SECRET		Secret for signing access tokens (min 32 chars)
JWT_EXPIRES_IN		Access token TTL (e.g. 15m)
JWT_REFRESH_SECRET		Secret for signing refresh tokens (min 32 chars)
JWT_REFRESH_EXPIRES_IN		Refresh token TTL (e.g. 7d)

API Endpoints

Auth — /auth

Method	Endpoint	Auth	Description
POST	/auth/register	Public	Register a new staff user
POST	/auth/login	Public	Login — returns <code>accessToken</code> + <code>refreshToken</code>
POST	/auth/refresh	Bearer (refresh)	Issue a new token pair
POST	/auth/logout	Bearer (access)	Invalidate refresh token

Services — /services

Method	Endpoint	Auth	Description
GET	/services	Public	List active services (search + pagination)
GET	/services/:id	Public	Get a single service by ID
POST	/services	Bearer	Create a service
PATCH	/services/:id	Bearer	Update service details
DELETE	/services/:id	Bearer	Soft-delete (sets <code>isActive = false</code>)

Bookings — /bookings

Method	Endpoint	Auth	Description
POST	/bookings	Public	Create a booking
GET	/bookings	Bearer	List all bookings (filter + pagination)
GET	/bookings/:id	Bearer	Get a single booking
PATCH	/bookings/:id/status	Bearer	Update booking status
DELETE	/bookings/:id	Bearer	Cancel booking (sets status to CANCELLED)

Business Rules

1. **Active service required** — bookings are rejected with 404 if the service does not exist or is inactive.
2. **No past dates** — bookingDate must be today or in the future (400).
3. **No double-booking** — the same service/date/time slot can only be booked once (409). Enforced at both the application layer (pre-check) and database layer (unique constraint).
4. **Status transition guard** — CANCELLED → COMPLETED is blocked with 400.

Swagger Documentation

Visit <http://localhost:3000/api/docs> after starting the server.

Quick auth flow in Swagger:

1. POST /auth/register → create your account
2. POST /auth/login → copy the accessToken from the response
3. Click **Authorize** (top right) → paste the token → **Authorize**
4. All 🔒 endpoints are now unlocked

Running Tests

```
# Run all unit tests
npm test

# Watch mode
npm run test:watch

# Coverage report
```

```
npm run test:cov
```

Current coverage:

- AuthService — 7 tests (register, login edge cases + happy paths)
 - BookingsService — 8 tests (past date, duplicate slot, status transitions, cancel)
-

Database Schema

```
users          id (uuid) | name | email (unique) | password (bcrypt) |
refreshToken   (hashed)
services       id (uuid) | title | description | duration (min) | price (decimal)
| isActive | createdBy (FK → users)
bookings       id (uuid) | customerName | customerEmail | customerPhone |
serviceId (FK → services)
               bookingDate (YYYY-MM-DD) | bookingTime (HH:MM) | status (enum) |
notes
               UNIQUE(serviceId, bookingDate, bookingTime)
```

Design Decisions & Assumptions

- **Refresh token hashing** — raw refresh tokens are never stored in the database; only a bcrypt hash is persisted to mitigate token theft from DB dumps.
 - **Soft delete** — services are deactivated (`isActive = false`) rather than hard-deleted to preserve booking history referential integrity.
 - **Date/time as strings** — `bookingDate (YYYY-MM-DD)` and `bookingTime (HH:MM)` are stored as `VARCHAR` to ensure the unique constraint (`serviceId, bookingDate, bookingTime`) behaves predictably across all PostgreSQL environments without timezone ambiguity.
 - **Decimal price** — `@db.Decimal(10, 2)` avoids floating-point rounding errors for monetary values.
 - **Public booking creation** — customers create bookings without authentication; staff log in to manage them.
-

How to Use (Visual Guide)

1. Registration & Login

Staff members can create an account and log in. Once logged in, copy the `accessToken`.

! [Registration and Login]()

2. Authorize

Click the **Authorize** button (padlock icon) at the top right of the Swagger UI. Paste your

accessToken to unlock protected endpoints.

![[Authorize]()]

3. Manage Services & Bookings

With authorization, staff can create new services (POST /services), view all bookings (GET /bookings), and update booking statuses (PATCH /bookings/:id/status).

![[Services and Bookings]()]

4. Overall Screenshot (Remember: Zoom in to see clearly)

Author

Built by **Shubham** as part of the EN2H Backend Engineering Internship Technical Assessment.